

Code:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/wait.h>
6
7 int main()
8 {
9     pid_t child1, child2, child3;
10     int status;
11
12     printf("Parent PID: %d\n", getpid());
13
14     // Create Child 1
15     child1 = fork();
16     if (child1 < 0)
17     {
18         perror("fork failed");
19         return 1;
20     }
21     if (child1 == 0)
22     {
23         printf("Child 1: PID = %d\n", getpid());
24         sleep(2);
25         printf("Child 1 completed.\n");
26         exit(10);
27     }
28
29     // Create Child 2
30     child2 = fork();
31     if (child2 < 0)
32     {
33         perror("fork failed");
34         return 1;
35     }
36     if (child2 == 0)
37     {
38         printf("Child 2: PID = %d\n", getpid());
39         sleep(4);
40         printf("Child 2 completed.\n");
41         exit(20);
42     }
43
44     // Create Child 3
45     child3 = fork();
46     if (child3 < 0)
47     {
48         perror("fork failed");
49         return 1;
50     }
51     if (child3 == 0)
52     {
53         printf("Child 3: PID = %d\n", getpid());
54         sleep(1);
55         printf("Child 3 completed.\n");
56         exit(30);
57     }
58
59     // Parent waits using wait()
60     printf("Parent using wait()...\n");
61     pid_t finished = wait(&status);
62     if (finished != -1)
63     {
64         printf("wait(): Child PID %d finished with status %d\n",
65               finished, WEXITSTATUS(status));
66     }
67
68     // Parent waits specifically for Child 2
69     printf("Parent using waitpid() for Child 2...\n");
70     waitpid(child2, &status, 0);
71     if (finished != -1)
72     {
73         printf("waitpid(): Child 2 PID %d finished with status %d\n",
74               child2, WEXITSTATUS(status));
75     }
76
77     // Wait for remaining child
78     waitpid(child1, &status, 0);
79     if (finished != -1)
80     {
81         printf("waitpid(): Child 1 PID %d finished.\n", child1);
82     }
83
84     printf("All required child processes completed.\n");
85     return 0;
86 }
```

Output:

```
Parent PID: 895

Child 1: PID = 896
Child 2: PID = 897

Parent using wait()...
Child 3: PID = 898
Child 3 completed.
wait(): Child PID 898 finished with status 30

Parent using waitpid() for Child 2...
Child 1 completed.
Child 2 completed.
waitpid(): Child 2 PID 897 finished with status 20
Child 1 PID 896 finished.

All required child processes completed.
```